UNIVERSIDAD PANAMERICANA

Ciencias Aplicadas

Sistemas y Tecnologías de la información y Comunicación

Programación III

Laboratorio No. 2

Marcos Naasón Monzón Zacarias (000149995)

José Pablo Ibáñez Ibáñez (00014929)

Rebeca Abigail Oliva Valdez (000148781)

José Alejandro L´pez Cobo (000110691)

Ing. German Orlando Pérez Marroquin

Guatemala 13 de marzo del 2026

Planteamiento del problema

El ejercicio plantea una situación de concurrencia en una terminal de buses. Los pasajeros deben abordar de forma ordenada y el sistema debe garantizar dos restricciones simultáneas: primero, únicamente tres pasajeros pueden subir al bus al mismo tiempo; segundo, el bus no puede sobrepasar su capacidad máxima de treinta pasajeros.

Además, cada pasajero tarda entre dos y cinco segundos en completar el abordaje, porque debe validar su boleto y acomodar su equipaje. El programa debe simular múltiples pasajeros intentando abordar y utilizar semáforos para controlar tanto el acceso simultáneo como el límite total de pasajeros.

Interpretación técnica del enunciado

Desde el punto de vista de programación concurrente, cada pasajero se modela como un hilo independiente. Todos los hilos compiten por recursos compartidos: el área de subida y los cupos disponibles dentro del bus.

Eso significa que no basta con crear pasajeros en secuencia. Es necesario sincronizar los hilos para evitar condiciones de carrera, sobreventa de asientos o que más de tres pasajeros aborden de manera simultánea.

La solución final también incorpora entrada manual desde la terminal. Esto permite que el usuario indique cuántos pasajeros ya están a bordo al iniciar el programa y cuántos pasajeros nuevos intentarán abordar. Así se pueden probar escenarios reales, por ejemplo, comenzar con veinte pasajeros dentro del bus y simular la llegada del resto.

Objetivos de la solución

Objetivo	Explicación
Control simultáneo	Permitir que como máximo tres pasajeros suban al mismo tiempo.
Control total	Impedir que el bus supere la capacidad máxima de treinta pasajeros.
Rechazo automático	Cuando ya no haya cupo, los nuevos pasajeros deben ser rechazados.
Interacción manual	Permitir que el usuario defina desde la terminal el estado inicial del bus.
Simulación realista	Representar el tiempo de abordaje con una espera aleatoria entre dos y cinco segundos.

Estructura general del programa

La solución se divide en dos clases principales. La primera es la clase Laboratorio2, que contiene la lógica general del sistema, la lectura de datos desde la terminal, la configuración de semáforos y la creación de hilos. La segunda es la clase Pasajero, que representa a cada pasajero como un hilo y define el comportamiento individual de abordaje.

La estructura lógica puede resumirse en cinco pasos: leer datos iniciales, calcular espacios disponibles, crear pasajeros nuevos como hilos, controlar el abordaje con semáforos y mostrar un resumen final del proceso.

Variables y recursos compartidos

Elemento	Tipo	Función en el programa
CAPACIDAD_BUS	Constante entera	Guarda el valor 30, que representa la capacidad máxima del bus.
MAXIMO_SUBIENDO	Constante entera	Guarda el valor 3, que representa el máximo de pasajeros subiendo al mismo tiempo.
semaforoSubida	Semaphore	Limita el acceso concurrente a la zona de abordaje.
semaforoCapacidad	Semaphore	Representa los espacios disponibles restantes dentro del bus.
bloqueo	Object	Se usa con synchronized para proteger las secciones críticas y ordenar impresiones.
pasajerosAbordados	Entero	Lleva el conteo total de pasajeros que están a bordo.

Explicación completa del uso de semáforos

Semáforo de subida

El semáforo llamado semaforoSubida se inicializa con tres permisos. Cada vez que un pasajero va a comenzar el abordaje, intenta adquirir uno de esos permisos. Si hay permisos disponibles, puede subir; si no, debe esperar hasta que otro pasajero termine y libere su permiso. De esa forma se cumple exactamente la restricción del problema: solo tres pasajeros pueden subir al mismo tiempo.

Semáforo de capacidad

El semáforo llamado semaforoCapacidad no controla la zona de abordaje, sino la capacidad total del bus. Se inicializa con el número de espacios libres disponibles. Si el programa arranca con veinte pasajeros ya a bordo, entonces el semáforo se crea con diez permisos. Cada nuevo pasajero intenta reservar un espacio usando tryAcquire(). Si lo consigue, significa que todavía hay cupo. Si falla, el bus ya está lleno y el pasajero es rechazado.

Secciones críticas

Aunque los semáforos controlan el flujo principal, todavía existen variables compartidas, como el contador pasajerosAbordados y la salida por consola. Por esa razón se utiliza synchronized sobre un objeto llamado bloqueo. Esto evita que dos hilos modifiquen el contador al mismo tiempo o que los mensajes en pantalla se mezclen de forma desordenada.

Paso a paso del algoritmo

1. El programa inicia y muestra un encabezado informativo.
2. Solicita al usuario cuántos pasajeros ya están a bordo. Ese valor debe estar entre 0 y 30.
3. Con base en ese dato, calcula los espacios libres restantes dentro del bus.
4. Inicializa el semáforo de capacidad con la cantidad exacta de espacios libres.
5. Solicita cuántos pasajeros nuevos intentarán abordar.
6. Crea un arreglo de hilos y genera un objeto Pasajero por cada nuevo pasajero.
7. Cada pasajero intenta reservar un espacio dentro del bus usando el semáforo de capacidad.
8. Si no hay espacio, el pasajero es rechazado inmediatamente.
9. Si sí hay espacio, el pasajero espera su turno para subir respetando el semáforo de tres permisos.
10. El hilo simula el tiempo de abordaje entre dos y cinco segundos.
11. Cuando termina, incrementa el contador de pasajeros a bordo y muestra el total actualizado.
12. Si el contador llega a treinta, el sistema informa que el abordaje se cerró automáticamente.
13. El hilo libera el permiso del área de subida para que otro pasajero pueda continuar.
14. Al final, el programa espera que todos los hilos terminen y muestra un resumen final.

Explicación de la entrada manual desde la terminal

La mejora solicitada consiste en no dejar fijo el número de pasajeros. En lugar de eso, el programa pregunta primero cuántos pasajeros ya se encuentran dentro del bus. Luego pregunta cuántos pasajeros nuevos intentarán abordar. Esta decisión hace que el laboratorio sea más demostrable, porque permite probar distintos escenarios sin cambiar el código.

Por ejemplo, si el usuario ingresa 20 pasajeros iniciales y luego indica que llegarán 15 pasajeros nuevos, el sistema solo permitirá que suban 10, porque esos son los espacios que quedan disponibles. Los otros 5 serán rechazados. Esto evidencia de manera muy clara el funcionamiento del semáforo de capacidad.

Explicación detallada de las partes del código

Importaciones

Se importan Scanner para la lectura desde teclado, Random para generar tiempos aleatorios y Semaphore para implementar la sincronización de hilos.

Clase principal Laboratorio2

Aquí se declaran las constantes, los semáforos, el objeto de bloqueo y el contador global. También se encuentra el método main, que organiza el flujo del programa.

Lectura y validación de datos

El programa valida que la cantidad inicial de pasajeros esté entre 0 y 30 y que el número de pasajeros nuevos no sea negativo. Esta validación evita entradas inválidas desde la terminal.

Creación del semáforo de capacidad

Una vez conocidos los pasajeros iniciales, el programa calcula cuántos lugares quedan y con ese valor crea el semáforo de capacidad. Esto es importante porque el número de permisos ya no es fijo, sino dinámico.

Creación de hilos

Cada pasajero nuevo se representa como un hilo independiente. Los hilos se crean dentro de un arreglo para poder esperar su finalización usando join().

Método run de la clase Pasajero

Dentro de este método se define toda la conducta del pasajero: reservar espacio, esperar turno, simular tiempo de subida, actualizar el contador y liberar el permiso del área de abordaje.

Uso de tryAcquire

Se usa tryAcquire sobre el semáforo de capacidad porque el pasajero no debe esperar indefinidamente por un asiento. Si no hay cupo, el rechazo debe ser inmediato.

Bloque finally

El permiso del semáforo de subida se libera dentro de finally para asegurar que, incluso si ocurre una interrupción, otro pasajero pueda continuar el proceso.

Ejemplo de ejecución explicado

Dato ingresado	Interpretación del sistema
Pasajeros ya a bordo: 20	El bus comienza con veinte lugares ocupados.
Pasajeros nuevos: 15	Quince hilos nuevos intentarán abordar.
Espacios disponibles: 10	El semáforo de capacidad tendrá diez permisos.
Resultado	Suben diez pasajeros nuevos y cinco son rechazados.

En ese escenario el programa se comporta de forma coherente: aunque quince pasajeros intenten abordar, solo diez logran reservar espacio. Luego, de esos diez, el semáforo de subida organiza el abordaje para que nunca haya más de tres pasajeros subiendo de forma simultánea.

Instrucciones para compilar y ejecutar

```
javac Laboratorio2.java
java Laboratorio2
```

Después de ejecutar el programa, la terminal solicitará los dos datos iniciales. A partir de ellos comenzará la simulación concurrente.

Conclusión

La solución del Laboratorio 2 modela correctamente un problema clásico de sincronización. Los semáforos permiten separar dos necesidades distintas: controlar cuántos pasajeros suben al mismo tiempo y controlar cuántos pasajeros caben en total dentro del bus. Al integrar entrada manual desde la terminal, el programa se vuelve más flexible y más útil para demostración académica.

En términos de aprendizaje, este ejercicio muestra que la concurrencia no consiste únicamente en crear varios hilos, sino en coordinar su acceso a recursos compartidos. Precisamente por eso los semáforos resultan fundamentales en esta clase de problemas.